



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. március 1.

Tartalom I

1 Információk

- Elérhetőségek
- Követelmények
- Tervezett tanmenet

2 Bevezetés (algoritmus)

- Algoritmusok fontossága
- Műveletigény
- Tárhely igény

3 Adatszerkezetek és típusok

- Bevezetés
- Adattípus: Tömb

4 Rendezési probléma

Információk

- Oktató: Pusztai Gábor
- Fájlok: `Canvas` és `wornox.web.elte.hu`
- Email: `pusztaigabor@inf.elte.hu`
- Hasznos források:
 - Előadó anyagai:
 - `aszt.inf.elte.hu/~asvanyi/ad/ad1jegyzet/`
 - `people.inf.elte.hu/kinga/algorithmusok1/seged_ea.htm`
 - Könyv:
Új algoritmusok (Cormen-Leiserson-Rivest-Stein)
 - Könyv:
`https://tamop412.elte.hu/tananyagok/algorithmusok/index.html`
 - Animációk:
`https://people.inf.elte.hu/pgm6rw/algo/index.html`

- A személyes részvétel kötelező: legfeljebb 3 hiányzás engedélyezett
- **Canvas kvízek:**
 - Hetente megoldandó feladatok a tanultak kapcsán
 - Többször is kitölthető (kötelező házi feladat jellegű)
 - Minden kis kvízen legalább 70%-ot el kell érni
- **2 dolgozat** az órák során:
 - Félévközi dolgozat:
 - Mikor: körülbelül a 7. órán
 - Időpont és helyszín: ezen az órán
 - Formátum: papíron | Pontszám: 50 pont (minimum: 20)
 - Félévvégi dolgozat:
 - Mikor: körülbelül az utolsó órán
 - Időpont és helyszín: ezen az órán
 - Formátum: papíron | Pontszám: 50 pont (minimum: 20)

- **Szorgalmi** pontok:
 - A félév során lehet szerezni órai munkával vagy szorgalmi feladattal. (Maximum 10 pont)
- **Értékelés:**
 - A végső jegyet a két dolgozat összpontszáma határozza meg, mely szorgalmi ponttal javítható: $50+50(+10) = 110$ pont.
 - Mindkét dolgozatban **minimum 20** pontot kell elérni a teljesítéshez.
 - A félév végén lesz lehetőség a zárthelyi dolgozatok javítására/pótlására.
 - A javító zárthelyi eredménye felülírja a korábbi ZH eredményt.
- **Osztályzatok:**
 - (2) 40 - 54 pont
 - (3) 55 - 69 pont
 - (4) 70 - 84 pont
 - (5) 85 - 100/110 pont

- A szemeszter tervezett menete:
 - 1, Bevezetés; Buborék-, Kiválasztásos rendezés,
 - 2, Beszűrő-rendezés; Verem (Stack); Lengyel-forma
 - 3, Lista (1 irányú)
 - 4, Lista (2 irányú)
 - 5, Összefésülő rendezés (Merge sort)
 - 6, Gyors rendezés (Quick sort); Sor (Queue)
 - **7, Évközi dolgozat**
 - 8, Bináris fa
 - 9, Bináris keresőfa
 - 10, Kupac (Heap); Prioritásos sor; Kupacrendezés
 - 11, Hasító tábla
 - 12, Lineáris; Radix, Edény-, Leshámoló-rendezés
 - **13, Évvégi dolgozat**

Bevezetés (algoritmus)

Miért fontos algoritmusokat és adatszerkezeteket tanulni?

- Problémák megoldása véges számítási erőforrásokkal:
 - idő (*"Az idő pénz", "A pénzt visszaszerezheted, miután elköltötted, de az időt soha nem kaphatod vissza."*)
 - memória (*manapság olcsó, de még mindig nem végtelen*)
- Optimalizáció:
 - idő (*várakozási idő*)
 - memória (*pl. a felhőtárhely drága*)

Mi az algoritmus?

- ***Algoritmusnak** nevezünk bármilyen jól definiált számítási eljárást, amely bemenetként bizonyos értéket vagy értékeket kap és kimenetként bizonyos értéket vagy értékeket állít elő (véges időn belül).*
- *Eszerint az algoritmus olyan számítási lépések sorozata, amelyek a bemenetet átalakítják kimeneté.*
- *Az **algoritmusokat** tekinthetjük olyan eszköznek is, amelynek segítségével pontosan meghatározott számítási feladatokat oldunk meg.*

(Cormen-Lieserson-Rivest-Stein - Új algoritmusok - 1.1 Algoritmusok)

Műveletigény: Egy absztrakt időmérték. A program futása során megszámoljuk az alprogramhívásokat + a ciklusismétlések számát.

Jelölések:

- $MT(n)$ (Maximális idő),
- $AT(n)$ (Átlagos vagy várható idő)
- $mT(n)$ (Minimális idő)

$$MT(n) \geq AT(n) \geq mT(n)$$

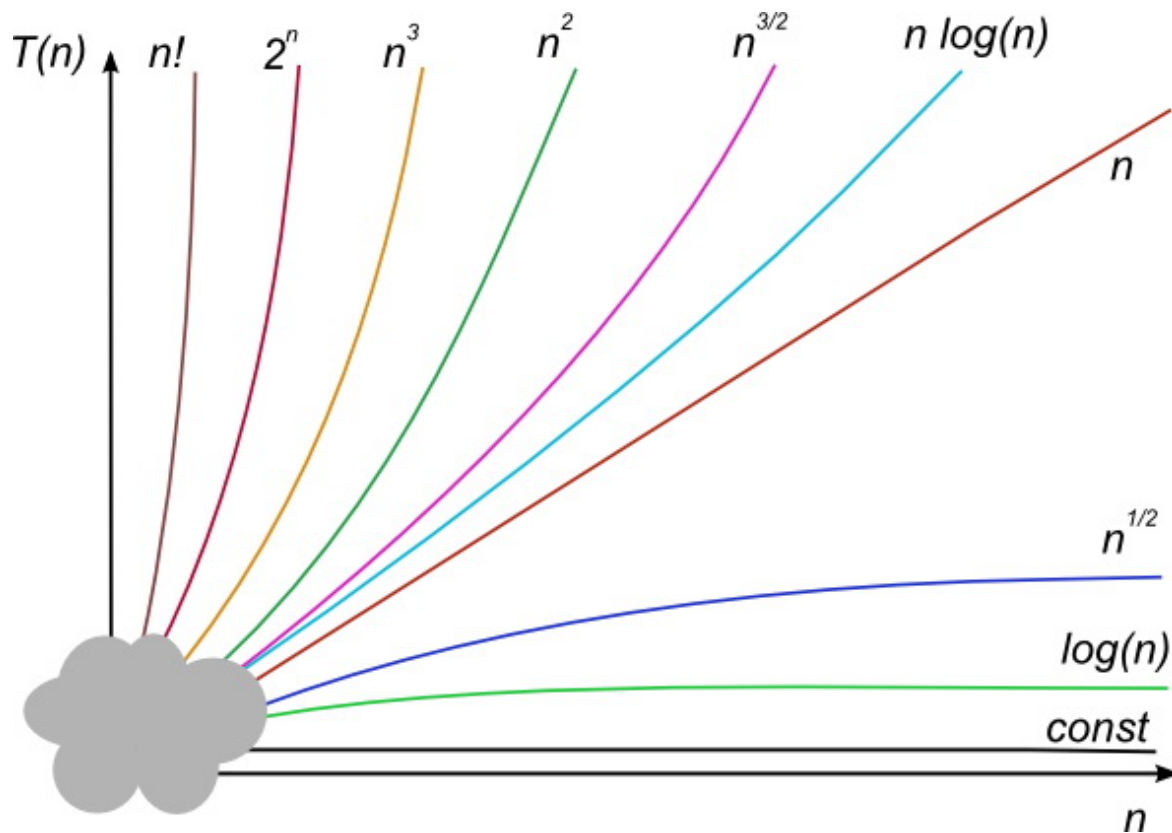
Az algoritmusok műveletigényét általában nem pontosan számítjuk ki, hanem csak *aszimptotikus rendet* vagy *aszimptotikus becslést* végzünk.

Aszimptotikus becslés(ek)

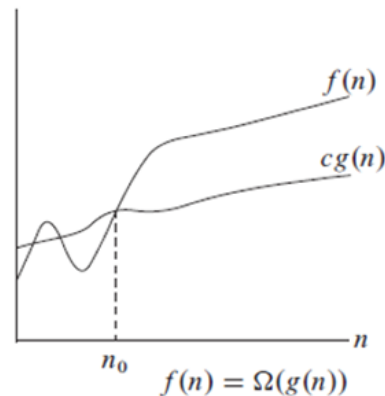
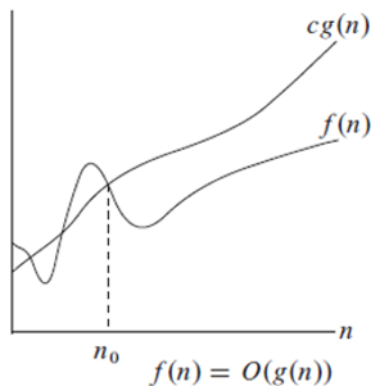
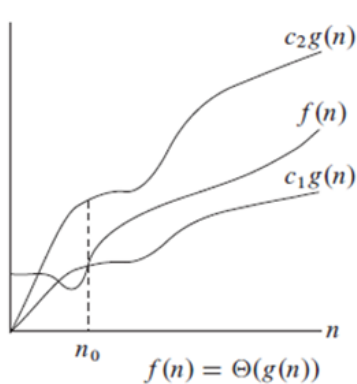
- Aszimptotikus FELSOR becslés:
 - O (big-O, Omikron)
 - a futási idő *legfeljebb** az adott érték
- Aszimptotikus ALSÓ becslés
 - Ω (Omega)
 - a futási idő *legalább** az adott érték
- Aszimptotikus FELSOR ÉS ALSÓ becslés
 - Θ (Theta)
 - a futási idő *"valahol az adott érték körül várható"**

**Pontosabb definíció az előadáson*

Aszimptotikus becslés(ek)



Aszimptotikus becslés(ek)



Polinom helyettesítési értékének kiszámítása

- Vizsgáljuk meg a ciklusiterációk $it(n)$, a szorzások $S(n)$ és az összeadások $\ddot{O}(n)$ számát, a polinom fokszámának függvényében.

Polinom1(Z:R[]; x:R) :R

y := Z[0]
i = 1 to Z.length-1
h := x
j = 1 to i-1
h := h * x
y := y + h * Z[i]
return y

Hányszor fut le (Z.length=n+1)

1
 $n+1$ (ciklusfeltétel kiértékelés)
 n
 $1+2+3+\dots+n-1+n$
 $0+1+2+3+\dots+n-1$
 n
 1

$$S(n) = \frac{n * (n + 1)}{2} = \frac{n^2 + n}{2}$$

$$\ddot{O}(n) = n \quad it(n) = S(n)$$

Rekurzív(Z:R[]; x:R) :R

y := Z[0] h := 1
i = 1 to Z.length-1
h := h * x
y := y + h * Z[i]
return y

Hányszor fut le (Z.length=n+1)

1
 $n+1$
 n
 n
 1

$$S(n) = 2 * n \quad it(n) = n$$

$$\ddot{O}(n) = n$$

Polinom helyettesítési értékének kiszámítása

Horner(Z:R[]; x:R) :R

y:=Z[Z.length-1]
i= Z.length-2 downto 0
y:=y*x+Z[i]
return y

Hányszor fut le (Z.length=n+1)

1
n+1
n
1

$S(n) = n$

$\ddot{O}(n) = n$

$it(n) = n$

	Polinom	Rekurzív	Horner
$S(n)$	$\theta(n^2)$	$\theta(n)$	$\theta(n)$
$\ddot{O}(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
$it(n)$	$\theta(n^2)$	$\theta(n)$	$\theta(n)$

Tárhely igény: Az algoritmus memóriaigényének absztrakt mértéke. Függ az alprogramhívások mélységétől és az adatszerkezetek méretétől.

Jelölések:

- $MS(n)$ (Maximális memóriaigény),
- $AS(n)$ (Átlagos vagy várható memóriaigény)
- $mS(n)$ (Minimális memóriaigény)

$$MS(n) \geq AS(n) \geq mS(n)$$

Adatszerkezetek és típusok

Adatszerkezetek és adattípusok

Adatszerkezet:

- adatok tárolásának és
- adatok elrendszerezésének egy lehetséges módja.

Miért használunk különböző adatszerkezeteket?

- különböző célokra → különböző optimális megoldások

A különböző adatszerkezetekhez különböző műveletekre van szükség.

Adattípus:

- egy adatszerkezet,
- a rajta értelmezett műveletekkel együtt.

Adattípus: Tömb

Tömb:

- A leggyakoribb adattípus
- Közvetlen hozzáférés az **adatokhoz** *indexelés* segítségével.
- Jelölések:

$$A : \mathcal{T}[n]$$

- A : Tömb
- T : Típus
- n : A mérete
 - $A.length$
 - Az A -ban tárolható adatok maximális mennyisége.

<i>Index:</i>	0	1	2	3	4
Data:	3	5	2	4	1

Rendezési probléma

Rendezési probléma

A félév során erre tanulunk különböző módszereket:
Egy számsorozatot növekvő sorrendbe kell rendezni!

Példa:

- bemenet: (31, 41, 59, 26, 41, 58)
- helyes kimenet: (26, 31, 41, 41, 58, 59)

Melyik algoritmus a legjobb egy adott alkalmazáshoz?

Sok tényezőtől függ, például:

- a rendezendő elemek száma,
- az elemek mennyire rendezettek már előre (pl.: 2,1,3,4,5)
- az elemek értékeire vonatkozó esetleges korlátozások (pl.: csak pozitív egész számok)
- hardverkonfigurációk

Miért érdemes "egyszerűbb" közvetlen módszereket tanulni a "legjobbak" helyett?

- Kezdők számára könnyebb megérteni.
- Könnyebben megvalósítható kódban.
- Kis n értékeknél (kis adatmennyiségnél) gyorsabbak lehetnek.

Köszönöm a figyelmet!